

ML Specification Report — playground-series-s6e2

Heart Disease Prediction • AIDE agent (20-step tree search, Claude Sonnet via shim)

Scores grounded exclusively in `facts_aide.json` for run `2-tentacled-pompous-junglefowl` ; pipeline configuration read from AIDE's own `best_solution.py` and `report.md` , step-level intent from `journal.json` , and task metadata from `competitions/playground-series-s6e2/config.yaml` .

Contents	
Heart Disease Prediction • AIDE agent (20-step tree search, Claude Sonnet via shim)	1
Overview	1
Data	2
Models & Architecture	3
Training procedures	4
The search trajectory	4
Training configuration	5
Inference procedures	6
Evaluation & Benchmarking	7

Overview

The task is to rank 270,000 patients by heart-disease probability (metric: ROC-AUC) on a synthetic dataset derived from UCI Heart Disease. This run was executed not by a hand-built pipeline but by **AIDE**, a tree-search coding agent: at each step it writes one complete, self-contained `solution.py` , executes it in a sandbox under a 30-minute wall clock, reads back either the printed metric or the Python traceback, and mutates the best-known solution accordingly. Over **20 steps** it produced **17 scored solutions and 3 crashes**, and its champion — discovered at **step 19, the very last step of the budget** — is a weighted blend of a randomized-search-tuned **CatBoost** and a fixed **LightGBM**, scoring **OOF ROC-AUC 0.95551** locally and **private LB 0.95516**, placing **1027 / 4371 (76.5th percentile)**.

On the leaderboard this is a three-way dead heat at the top of the lane: AIDE's private score ties the from-scratch agent's `mine_priv` to five decimals and edges the NVIDIA reproduce-agent by 0.00008; the score table nevertheless records `local_winner: mine` and `lb_winner: mine` .

Why it matters. Cardiovascular disease is a leading cause of death worldwide; risk-ranking from routine clinical features supports triage and preventive screening — as decision support, not diagnosis. The secondary interest here is methodological: this competition is a near-saturated problem, and it tests whether an autonomous search agent can find the last fraction of a thousandth of AUC that separates a competent baseline from a competitive submission.

Data

Purpose of Data. Predict whether a patient has heart disease (Presence vs Absence) — a binary classification Playground Series episode (S6E2). **Data Format** is clean **tabular CSV**, all columns numeric. **Data Volume**, per `config.yaml`, is **630,000 training rows / 270,000 test rows** with **13 predictive features** once `id` and the target are removed; AIDE's own report independently describes the input as "~630K rows".

Data Quality must be reported with an important caveat: **AIDE never ran a standalone EDA pass.** There is no `eda_summary.json`, no correlation table, no duplicate count, and no train↔test shift check anywhere in the run, because the agent's loop has no EDA stage — its entire understanding of the data is whatever it encodes directly into each `solution.py`. What can be read off the champion script is that AIDE inferred the data to be **complete**: the winning solution contains no imputation at all (no `fillna`, no `SimpleImputer`), which is only safe on a dataset with no missing values. It also correctly identified, from step 0 onward, that eight of the thirteen columns are integer-encoded categoricals rather than true continuous variables — `Sex`, `Chest pain type`, `FBS over 120`, `EKG results`, `Exercise angina`, `Slope of ST`, `Number of vessels fluro`, `Thallium` — and this single structural insight is what drove the whole search toward CatBoost. The **strongest EDA signals** cannot be reported: AIDE computed no feature-importance or correlation statistics that survive into any artifact, so the usual "Thallium dominates" narrative is simply absent from this lane. That is a real limitation of the agent, not an omission of this report.

Annotation Guidelines. The label is `Heart Disease` $\in \{\text{Absence, Presence}\}$, mapped in code to $\{0, 1\}$ with `{"Presence": 1, "Absence": 0}`. Submissions are **positive-class probabilities** scored by AUC — a ranking, not a hard 0/1 decision.

Feature Set. The champion uses the **13 raw columns unchanged**, held in two parallel representations so each member of the blend sees the encoding it prefers:

Group	Content
Continuous, both members	<code>Age</code> , <code>BP</code> , <code>Cholesterol</code> , <code>Max HR</code> , <code>ST depression</code>
Categorical → CatBoost	the 8 columns above cast to <code>str</code> and declared via <code>cat_features</code> (ordered target statistics)
Categorical → LightGBM	the same 8 columns cast to pandas <code>category</code> dtype (native categorical splits)
Engineered	none in the champion

AIDE did try engineered features — cholesterol-per-age, BP-per-age, heart-rate reserve, `Max HR / (220 - Age)`, and a ST-depression $\times$ Slope-of-ST interaction — at steps 1, 9 and 13. Step 1 (XGBoost with one-hot plus ratios) scored 0.95512, below the plain LightGBM baseline; steps 9 and 13 both timed out before producing a score, so the hypothesis was never even cleanly measured on the strong blend. **No**

engineered-feature variant ever became the best node, and the agent eventually abandoned the axis entirely — the same verdict a controlled ablation would have reached, but arrived at by attrition rather than by design.

Splitting strategy. A single `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)`, hard-coded identically in every step from step 0 onward, so all 17 scored nodes are measured on the same scale and are directly comparable. The champion additionally nests an inner **3-fold search CV on a 35% random subsample** for hyperparameter selection, which keeps the tuning stage cheap but means the selected CatBoost hyperparameters were chosen on a smaller, noisier proxy than the folds they are finally scored on. The leaderboard was open: the run was submitted and carries **both a public and a private LB score**, so unlike a CV-only report this one can measure generalization directly.

Models & Architecture

Purpose of Architecture. A binary heart-disease classifier maximizing ROC-AUC. **Architecture Type** is a **two-member weighted blend of gradient-boosted decision trees** — a randomized-search-tuned CatBoost plus a fixed-hyperparameter LightGBM, combined by a single scalar weight fitted on out-of-fold predictions. There is no meta-learner, no stacking layer, and no third family: AIDE tested stacking (step 10) and a three-way blend with XGBoost (step 11) and kept neither.

Input Format is a 13-column mixed matrix with **Input Dimension 13**; a 1-D feature vector with no spatial or sequential structure. The two members ingest it differently — CatBoost via a `Pool` with eight declared categorical columns, LightGBM via `category`-dtype columns — which is the deliberate source of the blend's diversity.

Architecture Description. The CatBoost member runs up to `iterations=2000` with `early_stopping_rounds=100`, `loss_function="Logloss"`, `eval_metric="AUC"`, `random_seed=42`, `use_best_model=True`, and five hyperparameters selected by search: `depth`, `l2_leaf_reg`, `learning_rate`, `bagging_temperature`, and `border_count`. The search itself is an **8-candidate grid** — depths in {5, 6, 7, 8}, `l2_leaf_reg` in {2, 3, 4, 5, 6, 8}, `learning_rate` in {0.025, 0.03, 0.035, 0.04, 0.05, 0.06}, `bagging_temperature` in {0.0, 0.2, 0.3, 0.5, 0.8, 1.0}, `border_count` in {128, 254} — each evaluated with `iterations=800` and `early_stopping_rounds=60` under the inner 3-fold / 35%-subsample protocol. The LightGBM member is not tuned: it keeps `n_estimators=2000`, `learning_rate=0.03`, `num_leaves=31`, `max_depth=-1`, `subsample=0.8`, `colsample_bytree=0.8`, `reg_lambda=1.0`, `early_stopping(100)` — the same settings AIDE wrote at step 0 and never revisited.

Model Complexity is best read as trees $\times$ leaves rather than a dense parameter count: per fold, up to 2000 CatBoost trees at the selected depth plus up to 2000 LightGBM trees at 31 leaves, each truncated by early stopping, over 5 folds and 2 members. The **blend layer** is a single scalar: the weight w on the CatBoost OOF vector is swept over `np.arange(0.0, 1.01, 0.05)` — 21 candidates — and the value maximizing OOF AUC is applied unchanged to the fold-averaged test predictions.

Training procedures

The search trajectory

This is the section that distinguishes an AIDE report from a hand-built pipeline report. AIDE does not execute a planned ladder; it hill-climbs. Of the **20 steps** in the budget, **17 produced a score and 3 crashed**:

```
20 - 17 = 3 buggy steps
```

All three failures were the same exception, `TimeoutError (buggy_exc_types: {"TimeoutError": 3})`, and they fall at the three steps absent from the metric trajectory — **steps 9, 11 and 13**. Each hit the harness wall of 1800 s (equal to the recorded `exec_time_max_s`), so the timeouts alone consumed

```
3 × 1800 = 5400 s
5400 / 11931.9 ≈ 0.4526 → 45.3% of all execution time
11931.9 / 3600 ≈ 3.31 h total
```

Nearly half the compute budget of this run bought nothing. The cause is legible in the plans: step 9 tried rank-averaging plus five new engineered features on top of the blend, step 11 added a third 2000-round XGBoost member and widened the weight search from a scalar to a 3-way grid, and step 13 re-attempted the engineered features on the strongest blend — all three stacked 5-fold training of multiple 2000-iteration boosters over 630k rows inside a 30-minute box. AIDE diagnosed this correctly and immediately: steps 12, 14 and 16 each open with an explicit root-cause statement ("the timeout was caused by training three separate high-iteration models...") and each proposes a concrete budget cut — fewer models, fewer folds, fewer iterations, higher learning rate. The recovery worked mechanically but cost accuracy: the 3-fold repair at step 14 scored 0.95531 and the LightGBM-only repair at step 16 scored 0.95518, both below the 5-fold blend they were retreating from.

The metric trajectory itself is a story of one early jump followed by a long plateau:

Step	What AIDE changed	OOF AUC
0	LightGBM, raw integer-encoded features (baseline)	0.95527
1	XGBoost, one-hot + engineered ratio features	0.95512
2	LightGBM + Logistic Regression 50/50 blend	0.95462
3	LightGBM, 3-seed multi-seed bagging	0.95535
4	CatBoost , native ordered-target-statistic categoricals	0.95545
5	Random Forest	0.95147
6	PyTorch MLP with categorical embeddings	0.95311
7	Extra Trees	0.9517
8	CatBoost + LightGBM blend, grid-searched scalar weight	0.95546

Step	What AIDE changed	OOF AUC
9	rank-averaging + engineered features	TimeoutError
10	stacking: LR meta-learner on the two OOF vectors	0.95546
11	third member (XGBoost) + 3-way weight grid	TimeoutError
12	timeout repair: two members, fewer iterations, tighter ES	0.95541
13	engineered features on the best blend	TimeoutError
14	timeout repair: 3 folds, higher LR	0.95531
15	restore 5 folds, higher LR / fewer iterations	0.95545
16	timeout repair: drop CatBoost, LightGBM only	0.95518
17	search <code>./input</code> for the original UCI dataset as extra rows	0.95541
18	3-seed bagging of the blend	0.95543
19	randomized hyperparameter search on CatBoost + blend	0.95551

The whole search moves inside a band of

$$0.95551 - 0.95147 = 0.00404$$

and the productive part of it is far narrower still. From the first scored solution to the champion, AIDE gained

$$0.95551 - 0.95527 = 0.00024$$

— that is the entire return on 20 steps and 3.31 hours. Eight of the seventeen scored nodes land at or above 0.9554, which is a fair description of a saturated problem rather than an agent failure; but it is also true that AIDE spent steps 5, 6 and 7 (Random Forest, MLP, Extra Trees) confirming what the first four steps had already implied, and that the single highest-value move of the run — actually tuning the strongest member's hyperparameters — was not attempted until step 19, when there was no budget left to build on it. The best node being the last node is the clearest available evidence that the search had **not** converged: it was still improving when the step cap stopped it.

One source conflict is worth flagging. AIDE's own `report.md` tabulates the stacking experiment (step 10) at a noticeably lower figure than the journal records for that step, and lists rows that do not map one-to-one onto the 17 scored nodes. Where the two disagree, `facts_aide.json` — extracted mechanically from the journal — is authoritative, and it is what every number in this report uses.

Training configuration

The **Loss Function** is binary log-loss for both members (`loss_function="Logloss"` in CatBoost, the default `binary` objective in LightGBM), with model selection and early stopping driven by **AUC** (`eval_metric="AUC"` , `eval_metric="auc"`). No class weighting was applied anywhere in the champion, which is the correct choice for a ranking metric on a mildly imbalanced target — though AIDE arrived

there by never testing the alternative rather than by ablating it. The **Optimization Algorithm** is gradient boosting — CatBoost's ordered boosting with symmetric trees, LightGBM's histogram-based leaf-wise growth — **not** SGD/ADAM; the blend weight is optimized separately by exhaustive 21-point grid search on the OOF matrix. The **Learning Rate** is the boosting shrinkage: 0.03 fixed for LightGBM, and for CatBoost whichever of {0.025, 0.03, 0.035, 0.04, 0.05, 0.06} the inner search selected. There is **no Learning Rate Scheduler** (N/A — GBDT convergence is governed by CV early stopping, not a schedule) and **no Batch Size** (N/A — full-dataset histogram boosting, not mini-batched).

Training Duration across the whole search was **11,931.9 s** of execution time over 20 nodes, mean **596.6 s** per node, max **1,800.0 s** (the harness cap). Per-node wall time for the champion specifically is not carried in `facts_aide.json` and is therefore **not recorded** here. **Training Memory** was never instrumented — **not recorded**; a $630k \times 13$ matrix held twice (CatBoost string copy and LightGBM categorical copy) is comfortable on the single arm64 CPU machine, but no peak figure exists.

Transfer Learning is N/A (no pretrained weights), and — unlike a from-scratch agent with a persistent experience library — **AIDE has no cross-competition memory at all**: every run starts cold. Its only analogue is within-run memory, and that analogue is genuinely load-bearing here: the step plans quote earlier nodes' scores verbatim ("improve on the best-performing CatBoost solution (0.95545)", "the CatBoost+LightGBM 5-fold blend (0.95546) has been the best-performing approach so far"), which is how the search stays anchored to its incumbent instead of drifting. **Data Augmentation** is N/A (tabular); the analogues AIDE reached for were feature engineering (rejected, see Data) and external-data ingestion — steps 9 and 17 both hunted for the original UCI Heart Disease table in `./input` to use as extra training rows, and neither found it.

Reproducibility Standards are weaker than a hand-built pipeline's and should be stated plainly. A single `seed = 42` is threaded through the fold split, CatBoost's `random_seed`, LightGBM's `random_state`, and the `np.random.RandomState` that draws the search subsample, so the run is seed-consistent. But the champion script sets **no LightGBM determinism flags** — no `deterministic=True`, no `force_row_wise=True`, no pinned `num_threads` — so bit-identical reproduction across machines or thread counts is not guaranteed. There is also **no OOF-rebuild gate**: AIDE has no verification stage that retrains the champion and checks the cached predictions match. The submitted artifact is simply whatever `working/submission.csv` the winning node wrote, which for this competition was re-materialized by a direct rerun of the best node (`facts_aide.json` leaderboard message: "20-step best node (direct node rerun), local 0.95551").

Inference procedures

Decision Threshold is N/A — AUC is a ranking metric, so raw positive-class probabilities are submitted and never thresholded. **Post-processing** is likewise absent: no clipping, no rank transform, no calibration. Test predictions are accumulated during the same CV loop that produces the OOF vectors — each fold's model contributes `predict_proba(X_test)[: , 1] / 5` to its member's test vector — and the two member vectors are combined with the OOF-optimal weight `best_w` and written straight to

`submission.csv` against the `sample_submission` column layout. Note the consequence: the shipped model is a **5-fold ensemble, not a refit-on-all-data model**, so every test prediction is an average over five models each trained on 80% of the data. **Inference Duration** and **Inference Memory** were not profiled — **not recorded**.

Evaluation & Benchmarking

Task for Performance Evaluation. Rank the 270,000 test patients by heart-disease probability.

Performance Metrics. AIDE's champion scores **OOF ROC-AUC 0.95551** locally, **0.95364** on the public leaderboard and **0.95516** on the private leaderboard, for **rank 1027 / 4371** = the **76.5th percentile**. The local-to-private gap is small and in the expected direction:

```
0.95551 - 0.95516 = 0.00035    (local OOF above private LB)
0.95516 - 0.95364 = 0.00152    (private LB above public LB)
```

The 5-fold OOF estimate is therefore honest to about three ten-thousandths of AUC, and the public LB is the pessimistic of the three — a reminder that the public split on this competition is small enough to be noisy and that selecting on it would have been the wrong call. The search itself gained

```
0.95551 - 0.95527 = 0.00024
```

over its own first solution, all of it from switching to CatBoost (step 4), adding the LightGBM blend (step 8), and finally tuning CatBoost (step 19).

Performance Benchmarking. `facts_aide.json`'s score table carries the private-leaderboard figures for all three lanes, which makes this the one competition in this batch with a complete three-way comparison:

Agent	Approach	Private LB AUC	Note
AIDE	20-step tree search → tuned CatBoost + LightGBM blend	0.95516	ties <code>mine_priv</code> at 5 dp
From-scratch agent (<code>mine_priv</code>)	—	0.95516	<code>local_winner: mine</code> , <code>lb_winner: mine</code>
NVIDIA (<code>nvidia_priv</code>)	—	0.95508	0.00008 behind

```
0.95516 - 0.95516 = 0.00000    (AIDE vs mine, private LB)
0.95516 - 0.95508 = 0.00008    (AIDE vs NVIDIA, private LB)
```

At the reported five-decimal precision AIDE and the from-scratch agent are **numerically indistinguishable on the private leaderboard**, and both sit a hair above NVIDIA; the score table's own verdict fields (`local_winner: mine`, `lb_winner: mine`) break the tie in favour of the from-scratch lane. The honest reading is that on a saturated, feature-poor tabular problem, twenty steps of autonomous

search reach essentially the same ceiling as a deliberately engineered ladder — the competition simply does not have another thousandth of AUC left in it for anyone. What separates the lanes is cost and confidence, not score: AIDE spent 3.31 hours of compute of which 45.3% was wasted on timeouts, produced no EDA, no ablations and no reproduction gate, and found its best solution on the last step of a budget that was clearly too short.